Small Door

Senior Data Engineer Case Study

By: Renaud Beaupre

Made with GAMMA

What We Built

Single-source dataset → purpose-built **dbt stack on DuckDB**. All four dashboard questions answerable from `appointments` with **no joins at query time.**

Model	Layer / Type	Purpose
stg_clinic_appointments	staging view	Type casts and column renaming only
`int_clinic_appointments`	intermediate view	Quality fixes, flags, and derived columns
date_spine	ref table	Date reference 2015–2030
`appointments`	marts table	OBT — primary BI surface
patient_summary	marts table	Patient-level aggregate (standalone)
`clinic_summary`	marts table	Clinic-level aggregate (standalone)
no_show_by_lead_time	analysis table	No-show × lead-time × SMS

Made with GAMMA

Data Quality

Principle: flag and retain in the intermediate layer; filter for polished simplicity in the mart. Quality flags are always available for audit queries. Stakeholders never encounter ambiguous or impossible values in the mart.

Issue	Volume	Intermediate	Mart
PatientId fractional (e.g. 93779.52927)	5 rows	Decimal stripped → full-length ID (9377952927)	Retained
Age = −1 (impossible)	1 row	`age_invalid = true`	Filtered out
Age = 0 (ambiguous)	3,539 rows	No flag — likely newborns	Retained
Age > 110	5 rows	No flag — biologically possible	Retained
Same-day walk-ins (midnight UTC artifact)	38,563 rows	`same_day_appointment = true`	Retained — valid appointments, 4.7% no-show rate
Genuinely negative lead time (appt before schedule)	5 rows	`lead_time_genuinely_negative = true`	Filtered out
Small clinics (n=1–2)	~5 clinics	No flag	Retained — volume filtering belongs at reporting layer
Disability encoded 0–4	2,241 rows with disability > 0	Keep as integer	Retained — ordinal severity, collapsing loses signal

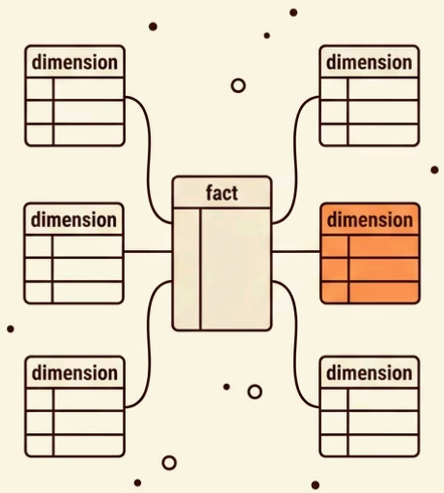
Made with GAMMA

Key Modelling Decisions

Decision	Chose	Rejected
Intermediate layer for quality logic	int_clinic_appointments owns all fixes, flags, and derived columns	Flags in staging; logic in mart
Flag and retain in intermediate; filter in mart	Bad rows flagged and kept in `int_clinic_appointments`; excluded from `appointments`	Drop at ingestion; expose flags to stakeholders
No dim_patient_history	Attributes carried directly from source row	SCD2 with BETWEEN join
`clinic_summary` mart	Dedicated clinic aggregate model	GROUP BY on `appointments` at query time
date_spine joined on date value	Direct equality join	Compute date attrs inline
`sms_sent` as boolean	Flag on OBT	`dim_sms` with one row per message
Disability as integer 0–4	Raw ordinal value	Cast to boolean
Clinic utilization proxy	`is_utilized = NOT no_show`	Capacity-based rate

Made with GAMMA

Hard Call 1 — Dashboarding: OBT vs Star Schema



Star Schema

Multiple sources

Needs SCD2 for patient history

Fan-out joins at query time

More models & tests

OBT chosen

Single table clinic_appointments

Each row carries state at scheduling time

Zero joins required

4 models total

Why OBT and Medallion?

Star schema solves an **integration problem** — joining entities across multiple source systems. There is no integration problem here: one source table, one grain, all dashboard questions answerable with **no joins** at query time.

On SCD2: each appointment row records patient state **at scheduling time** — that is the history. We built it and removed it. Deep-dive analysis shows only **3.2–3.9%** of consecutive appointment pairs have any attribute change between visits (only on age).

What would change our mind? Integration of a patient registration system and multiple upstream data domain sources which updates records outside of the appointments dataset.

Made with GAMMA

Hard Call 2 — Disability: Severity Scale or Specific Types?

Decision

Treat as an **ordinal severity scale** and retain as integer. Casting to boolean would irreversibly collapse the encoding.

Evidence for severity

- Distribution shows a sharp drop-off at higher values — most patients are 0, very few reach 3 or 4. Consistent with severity, where extreme values are naturally rare
- Hypertension and Diabetes rates increase with Disability level (sedentary lifestyle hypothesis)
- Disability = 2 patients show zeroes across all other comorbidity flags — rules out the hypothesis that Disability simply counts the other four flags.

What would change our mind: a data dictionary confirming specific type encoding.

Made with GAMMA

Hard Call 3 — Two Populations Inside "Negative Lead Time"

38,568 rows (~35%) have `scheduled_at > appointment_at` — but they are not one problem, they are two distinct populations.

Same-day walk-ins

38,563 rows. Same calendar date both sides, AppointmentDay always midnight UTC, **zero SMS reminders sent**, 4.7% no-show rate vs ~20% overall. → `same_day_appointment = true`, **retained in mart**

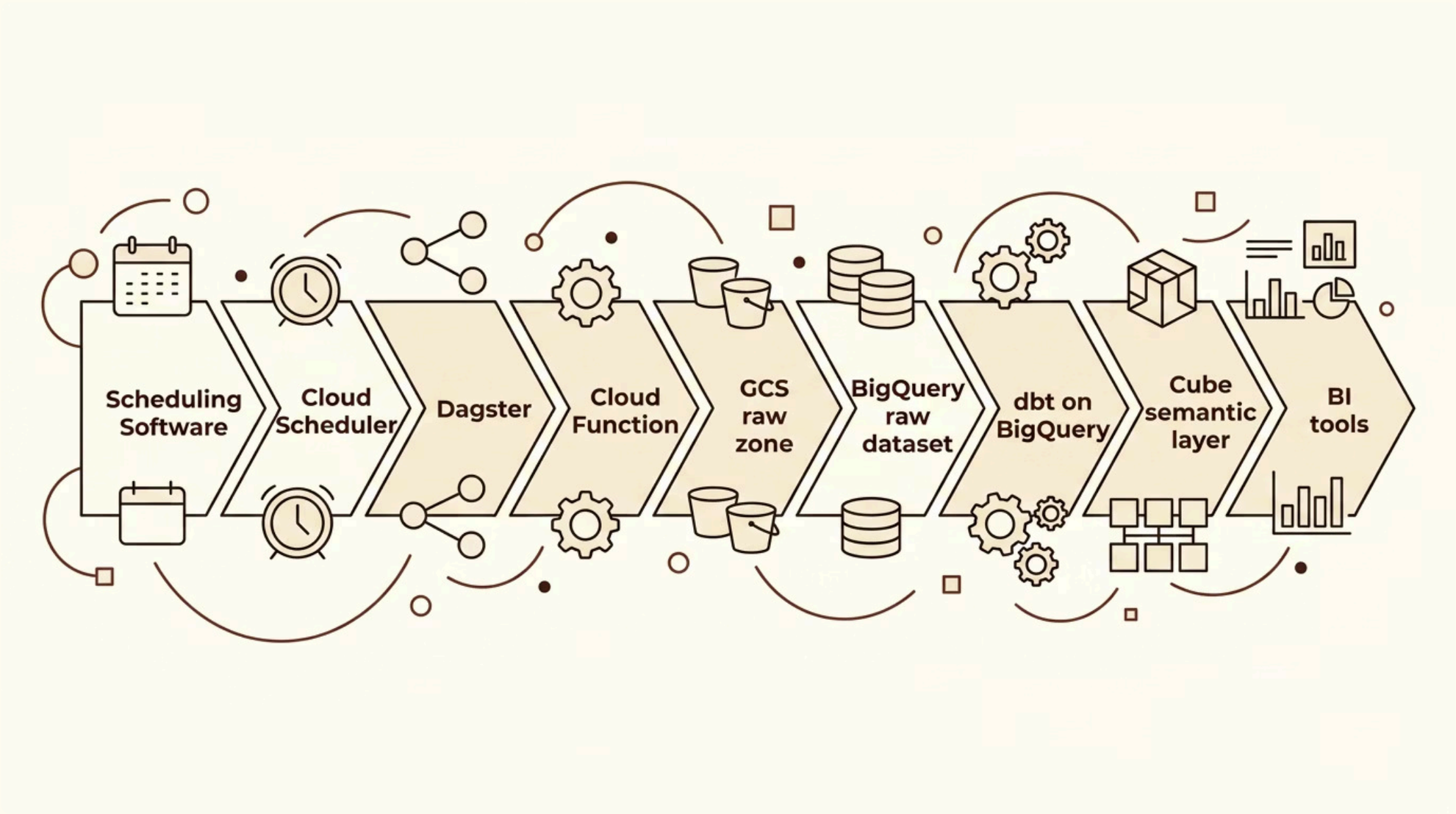
Genuinely negative

5 rows. Appointment date strictly precedes schedule date across calendar days. All are no-shows. → `lead_time_genuinely_negative = true`, **filtered from mart**

The risk of treating both as one: blending walk-ins with scheduled appointments without acknowledgement produces a **misleading overall no-show rate**. The two populations must be analysed separately.

Made with GAMMA

Production Architecture



What breaks first when source schema changes? `stg_clinic_appointments` — the only model that references source column names directly. A rename in the source requires a fix in one place. Dagster fails the staging asset and **blocks all downstream models automatically.**

Made with GAMMA

Testing & Freshness

How do you test that the numbers are right?

- **Ingestion:** Cloud Function publishes row count to Cloud Monitoring; alert on significant drop vs rolling average — catches silent truncations before they reach the warehouse
- **dbt:** unique, not_null, range checks run as Dagster asset checks — catches schema errors, constraint violations, out-of-range values
- **Dagster:** Row count assertions between raw and mart layers — catches row loss across the pipeline

How does an analyst know if today's data is stale?

`dbt source freshness` on `updated_at` as `loaded_at_field` — serves double duty as incremental watermark and freshness signal. If source is stale, **Dagster aborts** before any transforms run.

```
loaded_at_field: updated_at
freshness:
 warn_after: {count: 12, period: hour}
 error_after: {count: 25, period: hour}
```

Made with GAMMA

Metric Governance

Operations and Finance will want different definitions of **"no-show rate."** Solution: **Cube** as the semantic layer — all BI tools query through Cube, no direct BigQuery access.

Single access point

All BI tools query through Cube — no direct BigQuery access

Named measures

`no_show_rate` variants declared, version-controlled, peer-reviewed, one owner per definition

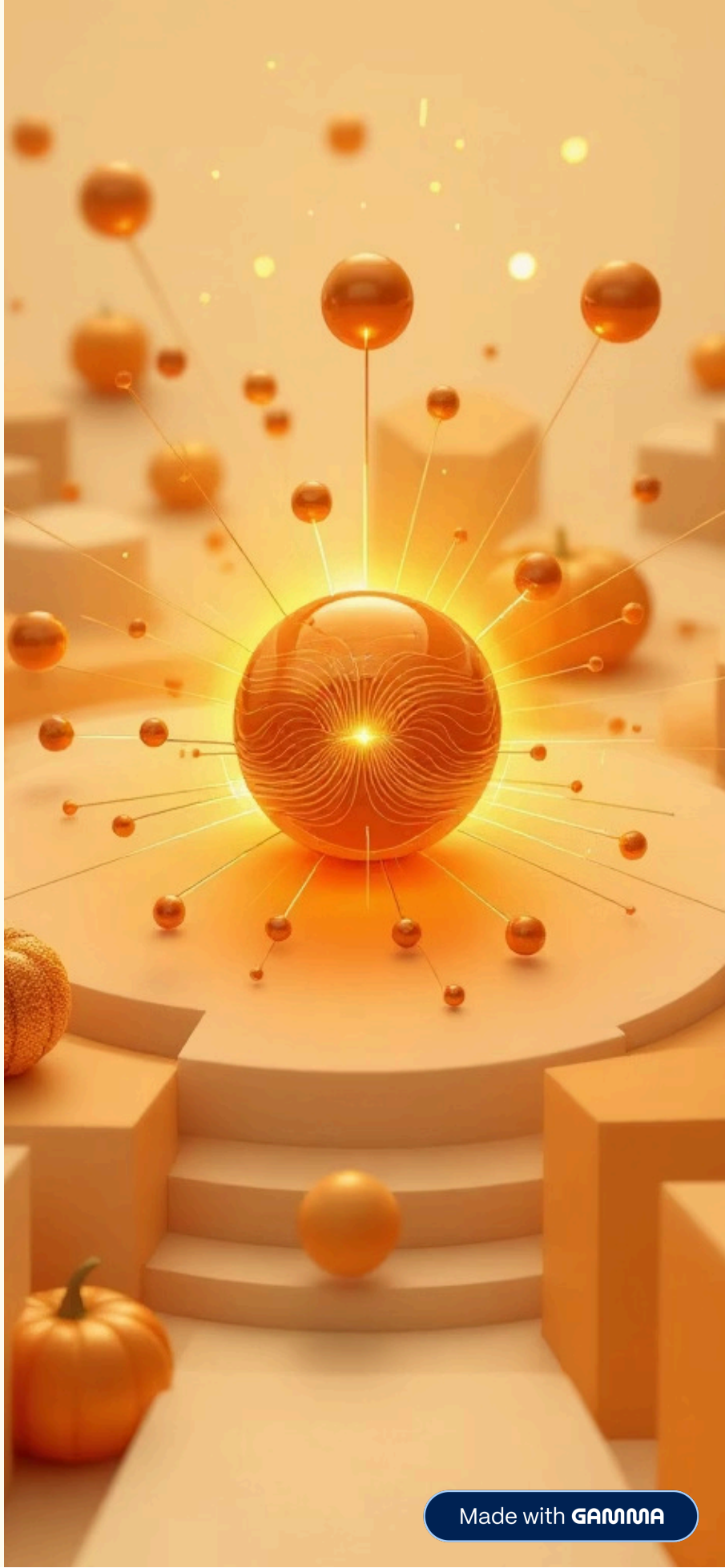
No shadow metrics

Analysts cannot invent a third definition in a calculated field

Domain separation

If definitions diverge at the data level, a separate dbt mart is built for that domain

The principle throughout: every metric definition has a **single owner, a written spec, and a version-controlled implementation** — never a spreadsheet or a BI tool calculated field.



Made with GAMMA